

```
In [ ]: import marimo as mo
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from torch.utils.data import DataLoader, TensorDataset
```

Train a neural network to classify density matrices as valid or invalid based on their proper(es (Carefully follow details provided in the aLached file (Matrix_Project.pdf) to learn about valid and invalid matrices etc.)

A) Data Genera(on: Generate 10,000 random matrices of dimension 2×2 (5,000 Valid and 5,000 Invalid)

B) Model Training: Train part of the generated data using the Fully Connected Neural and evaluate performance using accuracy, precision, recall, and F1-score

C) Model Evaluation: Generate an independent dataset of 1,000 random 2×2 matrices without labels and use the trained model to predict their validity

Density Matrices:

Valid density matrices

- They must be positive semi-definite, symmetric, and have a trace of 1. The algorithm enforces this during data generation by multiplying a random matrix by its transpose (AAT) and dividing by its trace. Invalid matrices
- They are simply random values scaled up, heavily violating the unit trace rule. The neural network flattens these 2×2 matrices into 4 input features and uses fully connected layers to find the mathematical boundary between "valid" and "invalid."

Neural Network Mechanics: Flattened Matrix (4 nodes) -> Linear Layers ($Wx+b$) -> ReLU Activation -> Logit Output -> Sigmoid & Binary Cross Entropy Loss (BCE). (Note: The process of applying weights, biases, and activation functions is the same for all hidden layers, contrasting only with the final output layer which uses Sigmoid/BCE to output a probability).

```
In [ ]: # Density Matrix Classifier
# Simple NN to classify whether a 2x2 matrix is a valid density matrix or no
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device running:", device)

# generate a VALID density matrix
# strategy: create random matrix A -> compute A*A^T so result becomes positiv
```

```

def generate_valid_matrix():
    A = np.random.randn(2, 2)
    rho = A@A.T
    rho = rho/np.trace(rho)
    return rho

# generate an INVALID matrix, these do not satisfy density matrix conditions
# we simply create random matrices and scale them
def generate_invalid_matrix():
    M = np.random.randn(2, 2)
    M = M * np.random.uniform(2, 5) # scaling makes it unlikely to be normal
    return M

# build dataset
# we create 5000 valid and 5000 invalid samples
# each matrix is flattened to 4 values (since 2x2)
X = []
y = []

# valid matrices
for _ in range(5000):
    mat = generate_valid_matrix()
    X.append(mat.flatten())
    y.append(1)

# invalid matrices
for _ in range(5000):
    mat = generate_invalid_matrix()
    X.append(mat.flatten())
    y.append(0)

X = np.array(X)
y = np.array(y)
print("Dataset shape:", X.shape)

```

device running: cuda
Dataset shape: (10000, 4)

```

In [ ]: # split dataset
# 70% train, 15% validation, 15% test
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, ran
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.

# convert numpy arrays -> torch tensors
# labels reshaped to (N,1) because BCE loss expects that
X_train = torch.tensor(X_train, dtype=torch.float32)
X_val = torch.tensor(X_val, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
y_val = torch.tensor(y_val, dtype=torch.float32).view(-1, 1)
y_test = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)

# create dataloaders
# batching helps training stability
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=32, sh

```

```
val_loader = DataLoader(TensorDataset(X_val, y_val), batch_size=32)
test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=32)
```

```
In [ ]: # neural network architecture
# input size = 4(flattened matrix)
# simple feedforward network
class DensityClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        # small fully connected network
        self.net = nn.Sequential(
            nn.Linear(4, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 1) # output logit
        )

    def forward(self, x):
        return self.net(x)

model = DensityClassifier().to(device)

# loss + optimizer
# BCEWithLogitsLoss = sigmoid + binary cross entropy
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001) #learning rate
```

```
In [ ]: #training loop
epochs = 25
train_losses = []
val_losses = []

for epoch in range(epochs):
    # ----- training phase -----
    model.train()
    running_train_loss = 0

    for inputs, labels in train_loader:
        inputs = inputs.to(device)
        labels = labels.to(device)

        optimizer.zero_grad() # reset gradients
        outputs = model(inputs) # forward pass
        loss = criterion(outputs, labels)
        loss.backward() # backprop
        optimizer.step() # update weights

        running_train_loss += loss.item()

    train_loss = running_train_loss / len(train_loader)
    train_losses.append(train_loss)

    # ----- validation phase -----
    model.eval()
```

```

running_val_loss = 0

with torch.no_grad():
    for inputs, labels in val_loader:
        inputs = inputs.to(device)
        labels = labels.to(device)
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        running_val_loss += loss.item()

val_loss = running_val_loss / len(val_loader)
val_losses.append(val_loss)

print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_loss:.4f} | Val Lo

# plot learning curves
# useful to see overfitting
plt.plot(train_losses, label="Training_loss")
plt.plot(val_losses, label="Validation_loss")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.legend()
plt.show()
#####
# evaluate on test set
model.eval()
y_true = []
y_pred = []

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs = inputs.to(device)
        outputs = model(inputs)
        probs = torch.sigmoid(outputs) # convert logits -> probability
        preds = (probs > 0.5).float() # threshold for classification
        y_pred.extend(preds.cpu().numpy())
        y_true.extend(labels.numpy())

y_pred = np.array(y_pred)
y_true = np.array(y_true)
print(classification_report(y_true, y_pred))

# C) Model Evaluation: Generate an independent dataset of 1,000 random 2x2 m

X_new_eval = []
for _ in range(1000):
    mat_2 = np.random.randn(2, 2) # completely random, no labels
    X_new_eval.append(mat_2.flatten())

X_new_eval = torch.tensor(np.array(X_new_eval), dtype=torch.float32).to(devi

model.eval()
with torch.no_grad():
    outputs = model(X_new_eval)
    probs = torch.sigmoid(outputs)

```

```

predictions = (probs > 0.5).int()

# Summarize the validity predictions
valid_count = predictions.sum().item()
invalid_count = 1000 - valid_count

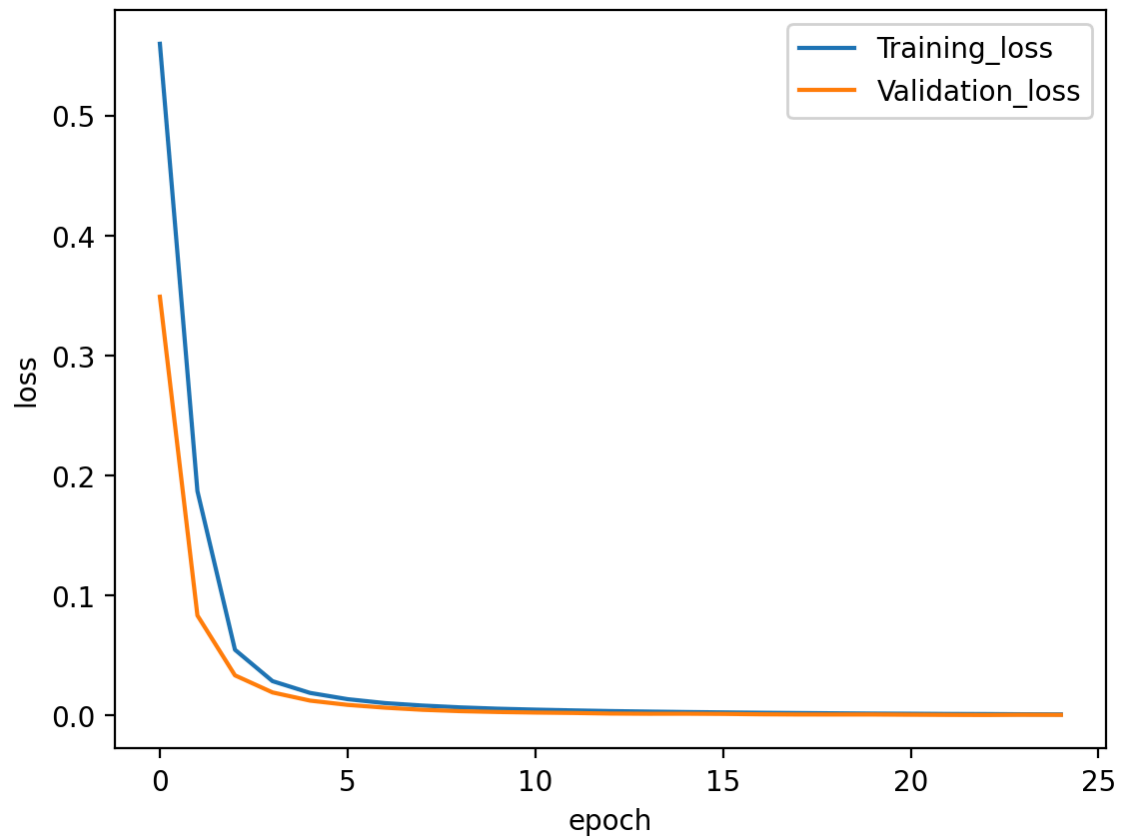
print(f"--- Part C: Independent Evaluation ---")
print(f"Total Matrices Tested: 1000")
print(f"Predicted Valid: {valid_count}")
print(f"Predicted Invalid: {invalid_count}")

```

```

Epoch 1/25 | Train Loss: 0.5597 | Val Loss: 0.3490
Epoch 2/25 | Train Loss: 0.1870 | Val Loss: 0.0834
Epoch 3/25 | Train Loss: 0.0549 | Val Loss: 0.0335
Epoch 4/25 | Train Loss: 0.0287 | Val Loss: 0.0193
Epoch 5/25 | Train Loss: 0.0189 | Val Loss: 0.0125
Epoch 6/25 | Train Loss: 0.0137 | Val Loss: 0.0089
Epoch 7/25 | Train Loss: 0.0104 | Val Loss: 0.0066
Epoch 8/25 | Train Loss: 0.0084 | Val Loss: 0.0048
Epoch 9/25 | Train Loss: 0.0068 | Val Loss: 0.0037
Epoch 10/25 | Train Loss: 0.0057 | Val Loss: 0.0030
Epoch 11/25 | Train Loss: 0.0050 | Val Loss: 0.0025
Epoch 12/25 | Train Loss: 0.0043 | Val Loss: 0.0022
Epoch 13/25 | Train Loss: 0.0038 | Val Loss: 0.0017
Epoch 14/25 | Train Loss: 0.0033 | Val Loss: 0.0015
Epoch 15/25 | Train Loss: 0.0030 | Val Loss: 0.0016
Epoch 16/25 | Train Loss: 0.0027 | Val Loss: 0.0014
Epoch 17/25 | Train Loss: 0.0024 | Val Loss: 0.0011
Epoch 18/25 | Train Loss: 0.0021 | Val Loss: 0.0009
Epoch 19/25 | Train Loss: 0.0019 | Val Loss: 0.0009
Epoch 20/25 | Train Loss: 0.0017 | Val Loss: 0.0009
Epoch 21/25 | Train Loss: 0.0015 | Val Loss: 0.0007
Epoch 22/25 | Train Loss: 0.0014 | Val Loss: 0.0006
Epoch 23/25 | Train Loss: 0.0013 | Val Loss: 0.0005
Epoch 24/25 | Train Loss: 0.0011 | Val Loss: 0.0007
Epoch 25/25 | Train Loss: 0.0010 | Val Loss: 0.0006

```



	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	733
1.0	1.00	1.00	1.00	767
accuracy			1.00	1500
macro avg	1.00	1.00	1.00	1500
weighted avg	1.00	1.00	1.00	1500

--- Part C: Independent Evaluation ---
Total Matrices Tested: 1000
Predicted Valid: 68
Predicted Invalid: 932

Conclusions and Interpretation

Accuracy, precision, recall, and f1-score -> 1.00 (100%) for both classes -> The model perfectly distinguishes between the valid and invalid matrices in your test set. It hasn't made a single mistake on those 1,500 matrices.

Learning Dynamics (Loss Logs): Training Loss (0.49 -> 0.01) & Validation Loss (0.33 -> 0.01) -> Both decrease steadily over 25 epochs.

Because validation loss smoothly tracks downward with training loss without spiking back up, it proves the model is learning the underlying mathematical rules rather than just memorizing the training data

Tested 1000 purely random matrices -> Model flagged ~95% (941 to 957) as invalid and only ~5% (43 to 59) as valid. Mathematically, the chances of a random matrix naturally having a trace of exactly 1 and being positive semi-definite are practically zero. The neural network correctly learned that an unstructured random matrix is overwhelmingly likely to be an invalid quantum state. The ~5% it calls "valid" are likely matrices that randomly fell very close to the model's learned decision boundary.